

מדריך טכני: עקרונות הארכיטקטורה הנקייה (Clean Architecture)

מסמך הסבר לדוגמה להקמת פרויקטים מודרניים בסביבת .NET ו-C#

1. מבוא ומושגי יסוד

ארכיטקטורה נקייה (Clean Architecture) היא דפוס עיצוב תוכנה השואף להפריד את הלוגיקה העסקית של האפליקציה (Core Business Logic) מהתשתיות הטכנולוגיות שלה (כגון בסיסי נתונים, ממשקי משתמש ומערכות חיצוניות).

היתרון המרכזי של גישה זו הוא יצירת קוד הניתן לבדיקה בקלות (Testable), קוד שאינו תלוי במסגרות עבודה חיצוניות (Framework Independent), וקל לתחזוקה לאורך זמן.

טיפ חשוב: כלל הזהב של ארכיטקטורה נקייה הוא חץ התלות (Dependency Rule). פרויקטים פנימיים לעולם אינם מכירים או תלויים בפרויקטים חיצוניים להם.

2. מבנה השכבות בפרויקט

מבנה הפרויקט הטיפוסי מחולק לארבע שכבות עיקריות:

- (ליבת המערכת) Domain:** מכיל ישויות (Entities), ממשקים, חוקים עסקיים בסיסיים וסוגי שגיאות ייחודיים. שכבה זו אינה תלויה בדבר.
- (שכבת היישום) Application:** מגדירה את תזרימי העבודה (Use Cases), פקודות (Commands) ושאלות (Queries). בדרך כלל נעשה כאן שימוש ב-MediatR.
- (תשתית) Infrastructure:** מימוש הממשקים שהוגדרו בשכבות הפנימיות, כגון חיבור ל-Entity. שליחת מיילים, ואינטגרציות חיצוניות Framework Core.
- Presentation / API:** נקודת הכניסה של המערכת (REST Controllers או Minimal APIs). שכבה זו אחראית על חשיפת הקצוות לעולם החיצון.

3. דוגמת קוד: הגדרת Entity ו-Repository

להלן דוגמה מעשית לכתיבת רכיבים בסיסיים בשכבת ה-Domain וה-Application בקוד C#:

```
// Domain Layer: Entity
public class PortalItem
{
    public Guid Id { get; private set; }
    public string Title { get; private set; }
    public DateTime CreatedAt { get; private set; }

    public PortalItem(string title)
    {
        Id = Guid.NewGuid();
        Title = title;
        CreatedAt = DateTime.UtcNow;
    }
}

// Application Layer: Interface
public interface IPortalRepository
{
    Task<PortalItem> GetByIdAsync(Guid id);
    Task AddAsync(PortalItem item);
}
```

4. השוואת ביצועים ומדדים

כאשר בוחנים את ההשפעה של חלוקה לשכבות על ביצועי המערכת, ניתן לראות את הנתונים הבאים:

פרמטר השוואה	ארכיטקטורה מונוליתית פשוטה	ארכיטקטורה נקייה (Clean)
זמן פיתוח ראשוני	מהיר מאוד	בינוני (דורש Setup)
קלות כתיבת טסטים (Unit Tests)	נמוכה / מורכבת	גבוהה מאוד (בידוד מלא)
עקומת למידה לצוות	נמוכה	בינונית ומעלה

5. ניתוח יעילות מתמטית של שאילות

בעת עבודה עם כמויות נתונים גדולות בשכבת התשתית, חשוב לוודא כי זמן הריצה של השאילות נשמר אופטימלי. אם נסמן את מספר הרשומות N ואת מספר האינדקסים, K , זמן הגישה הממוצע לרשומה בעץ חיפוש מאוזן מוגדר כ:

$$T(N) = O(K \cdot \log N)$$

שמירה על שכבת לוגיקה מופרדת מאפשרת להחליף את מנגנון השליפה (למשל מעבר ל-Redis Caching) ללא שינוי של אף שורת קוד בשכבת ה-Domain העסקית.